

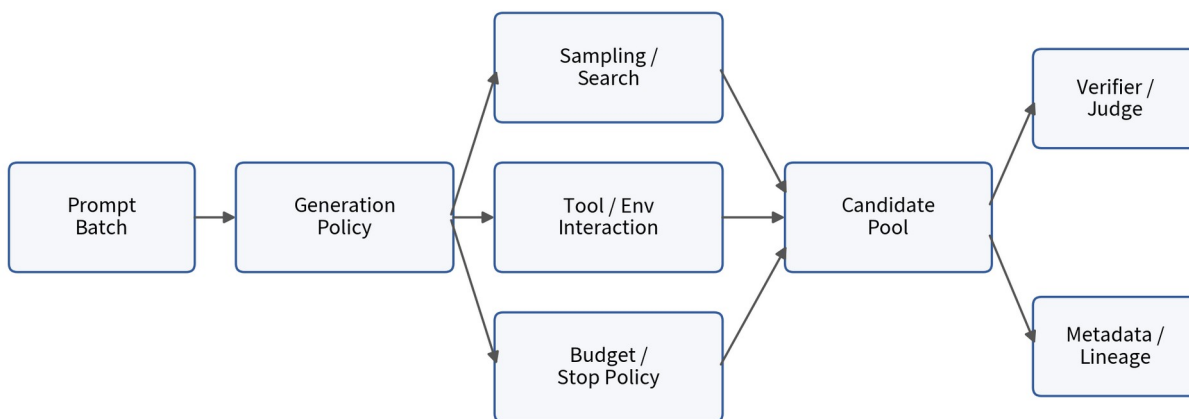
LLM 后训练之

Response Generation

从“解码”到 Experience Factory：候选响应、Rollout、搜索、轨迹与训练吞吐的系统工程

Response Generation = Experience Factory

控制模型在训练中“能看到多少种行为、轨迹和失败”



深度研究与工程手册

研究日期：2026-08-13 • 重点：Know-Why + Know-How • 适用：SFT / DPO / RLHF / RLVR / GRPO / Agent RL

一句话定义

Response Generation 是后训练中的“经验生成引擎”：它决定对每个 Prompt 采样多少候选、以怎样的随机性和预算探索、是否与工具/环境交互、如何停止，以及最终向 SFT / Preference / RL 暴露怎样的行为分布。

0. Executive Summary : 最重要的 12 个结论

- 不要把 Response Generation 等同于 temperature/top-p。它的工程对象是 Candidate / Trajectory Distribution，而不是单条文本。
- 后训练的“学习上限”受搜索空间限制：模型没有生成过的高质量行为，后续 Judge/Verifier 就无从选择，SFT/DPO/RL 也无从强化。
- Generation 有三种不同目标：造“金标准响应”（SFT）、造“可比较候选”（Preference/DPO）、造“可学习经验”（RL rollout）。三者的参数与质量标准不应共用。
- Best-of-N 的收益来自 breadth：样本越多，命中高奖励轨迹的机会越高，但收益递减且成本按 token/环境交互增长。[S1][S2]
- 温度不是固定超参，而是 policy 状态的一部分。Llama 2 观察到 RLHF 后最优采样温度会变化；DeepSeek-R1 不同 RL 阶段使用不同温度。[S1][S4]
- On-policy 数据通常更贴近当前 policy 的失败边界；Tulu 3 的公开实验显示，加入 on-policy preference responses 改善了聚合 DPO 表现。[S3]
- Reasoning RL 的“样本单位”已经从 response 变成 group of rollouts。DeepSeek-R1 / DAPO 典型每题采样 16 个输出，让组内 reward 方差变成学习信号。[S4][S5]
- 并非所有 Prompt 都值得同样多 rollout。DAPO 动态过滤组内全对/全错的 prompt，使 batch 保持有效梯度，是 Generation 与 Data Selection 融合的典型例子。[S5]
- 长度预算既是能力预算也是噪声源。过短截断复杂推理，过长会造成长尾、重复、reward noise 与训练吞吐下降。[S4][S5]
- Agent Response Generation 不只是“生成文本”，而是 reason → action/tool → observation → reason 的环境轨迹生成，因此环境延迟与失败恢复必须进入预算模型。
- 系统层面，rollout 正在成为 RL 后训练主要瓶颈。vLLM/PagedAttention、训练-推理解耦、异步 rollout、speculative rollout 都是在优化这条路径。[S9][S10][S11][S12]
- 成熟系统应该以“有效学习信号 / GPU-hour”而不是“responses/s”优化：高吞吐但低 reward variance、低 diversity、严重 staleness 的 rollout 并不一定有训练价值。

核心公式（工程抽象）

Effective Experience $\approx$ Coverage $\times$ Diversity $\times$ Correctness/Reward Signal $\times$ Freshness $\times$ Verifiability $\div$ (Token Cost + Environment Cost + Staleness Cost)

本手册的模块边界

上游	Response Generation（本手册）	下游
----	--------------------------	----

Prompt Factory / Task Distribution	policy / sampler / budget / rollout engine / env / stop rule / lineage	Judge / Verifier / Reward
Objective / Capability Spec	生成候选响应与轨迹的分布	Filtering / SFT / DPO / RL
模型 checkpoint / system prompt	on-policy/off-policy/teacher/student generation	Eval / Failure Mining / Data Flywheel

1. 第一性定：Response Generation 到底是什么？

在普通推理里，Response Generation 常被理解为“把 logits 解码成文本”；在后训练里，这个定义远远不够。后训练真正关心的是：从当前策略 π_θ 出发，在给定 Prompt 分布 D 上，构造一个可被评分、筛选、比较或优化的经验分布。

```
q ~ D_prompt
o_i ~ Generate( $\pi$ , q; temperature, top_p, max_tokens, system_prompt, tools, stop_policy)
 $\tau_i = (q, o_i, \text{tool\_calls}, \text{observations}, \text{logprobs}, \text{metadata})$ 
CandidatePool(q) = { $\tau_1, \dots, \tau_N$ }
```

因此 Response Generation 的直接产物不是“答案”，而是 Candidate Pool 或 Trajectory Pool。它既是模型能力的采样，也是训练数据的生产。



图 1 从 raw rollout 到 train-ready experience 的信号漏斗

1.1 为什么它决定训练上限？

Judge/Verifier 只能从“已经生成出来”的候选中做选择。若生成器始终走同一条低熵路径，即使 Judge 很强，也只能在一个贫瘠候选池中挑最好；相反，高熵但完全失控的生成器会制造大量无效 token。Response Generation 的任务是把搜索空间扩大到“足够覆盖高价值轨迹”，同时让成本与噪声可控。

Know-Why

训练算法负责“怎样更新参数”；Response Generation 决定“更新算法有机会看到什么行为”。这是 search space 与 optimization space 的接口。

1.2 四个输出层级

层级	产物	典型下游	关键问题
Single Response	1 条 completion	SFT / baseline	这条是否足够像 gold?
Candidate Set	N 条互相可比较的 responses	RM / DPO / Best-of-N	候选是否有质量差异与多样性?
Reasoning Rollout Group	N 条长 CoT + reward	GRPO / RLVR / DAPO	组内是否存在有效 reward variance?
Agent Trajectory	多轮 action-observation 序列	Agent SFT / Agent RL	是否真实完成环境任务?

2. Stage-Specific Generation : 不同训练阶段需要不同“候选分布”

阶段	Generation 目标	典型 N	随机性	选择/反馈	主要风险
SFT Teacher	高质量、可模仿的 target	1-多条	中低	Verifier / RM / 人工筛选	把 teacher 风格当能力
Rejection Sampling SFT	探索并选出高奖励 gold	10-30+	中高	RM / Judge 选 top	高分捷径、模式收缩
DPO/Preference	制造清晰的 chosen vs rejected	2-多模型池	中	Judge / human pairwise	候选太像，偏好信号弱
RLHF / PPO	在线采集当前 policy 经验	常 1/Prompt/step	policy-dependent	RM reward	昂贵、reward hack
GRPO / RLVR	问题多 rollout 构造相对优势	常 8-32	较高	规则/Verifier	全对/全错组无信号
Distillation	让 student 看见 teacher 的优质轨迹	1-多条	可控	correctness + quality	把错误或冗长轨迹蒸馏进去
Agent RL	探索 action/observation 序列	多 trajectory	动态	environment reward	环境成本、长尾、staleness

2.1 SFT : Response Generation 是“Target Factory”

对 SFT，首要目标不是最大 diversity，而是获得可信、可模仿、覆盖目标行为的响应。Tulu 3 的公开 recipe 中，新合成 SFT 数据的 response 由强 teacher 模型生成；这类 pipeline 应把“Prompt 是否值得训练”和“Response 是否正确”分开治理。[S3]

2.2 Preference : Response Generation 是“Contrast Factory”

Preference 数据需要有信息量的差异。候选如果几乎相同，Judge 很难给出稳定 preference；差异过大又容易让模型只学到粗粒度的长度/格式捷径。Tulu 3 会从多个模型中生成响应，并额外加入自身 SFT 模型的 on-policy 响应，再由 Judge 打分形成 chosen/rejected。[S3]

2.3 RL : Response Generation 是“Experience Factory”

RL 的 rollout 必须尽可能代表当前策略。此时 logprob、policy version、reward、stop reason 等都属于训练数据的一部分。Generation 不再是离线数据制作，而是训练循环内部的一等公民。

同样 1 个 Prompt，Response Generation 可以消耗完全不同的训练预算

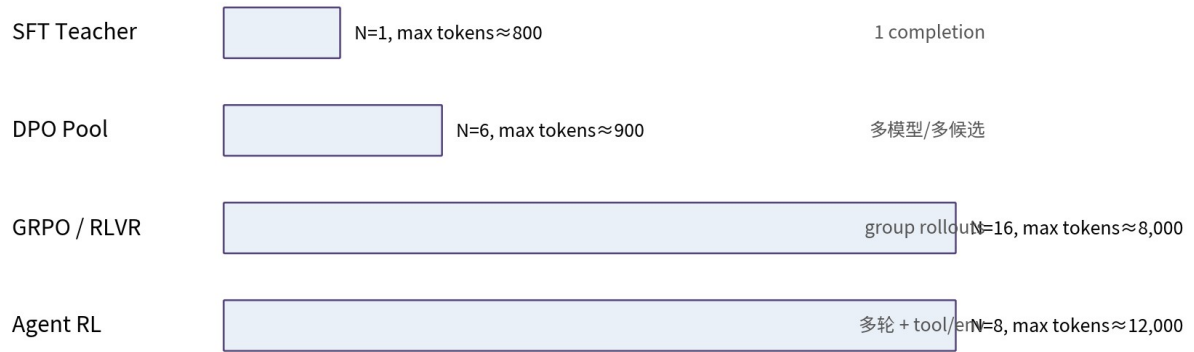


图 2 “同一个 prompt” 在不同后训练阶段的成本尺度完全不同（示意）

3. Sampling Primitives : 从 Greedy 到 Stochastic Search

3.1 基不是“味料”，而是在控制搜索空

参数	作用	过低/过小	过高/过大	后训练含义
temperature T	缩放 logits，控制随机性	模式集中、候选重复	不连贯/低质量增加	决定探索强度
top-p	只在累计概率质量内采样	过度保守	尾部 token 噪声	限制局部动作空间
top-k	只保留 k 个 token	削弱少见正确分支	k 大时接近无约束	控制分叉宽度
N / group size	每 prompt 候选数	错过稀有优质轨迹	成本与重复增加	Best-of-N / advantage 的 breadth
max tokens	单轨迹思考预算	截断推理	长尾与重复	能力预算 + 系统预算
stop rule	终止策略	过早终止	无效延长	影响 reward 与 token 成本

3.2 Temperature 需要随 Policy 重新校准

Llama 2 的 rejection-sampling 实验显示，更多样本会提高可获得的最大 reward，同时 temperature 影响探索；更重要的是，RLHF 迭代会改变最优 temperature。其报告中，RLHF 模型在 N=10-100 的设置下最优区间约为 1.2-1.3，并明确指出需要随着模型更新重新调整。[S1]

DeepSeek-R1 也体现了 stage-specific temperature：第一阶段 rollout 使用 T=1；第二阶段降到 T=0.7，因为更高温度在该阶段会导致不连贯生成。[S4]

Know-How

不要把 generation_config.yaml 当常量。把它版本化为 policy-version 的一部分，并在每轮训练后重新做 $N \times T \times \text{max_tokens}$ 的小规模网格评测。

3.3 Diversity 不是越高越好

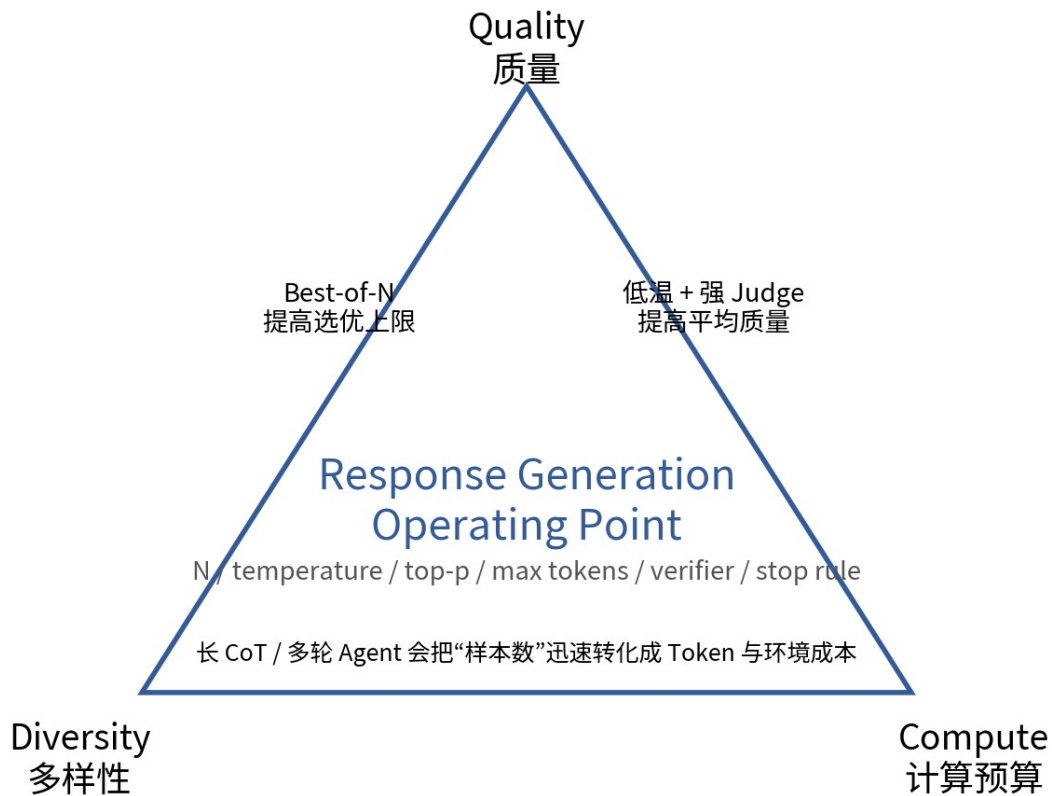


图3 Response Generation 的质量-多样性-计算预算三角

理想候选池需要“可用差异”：答案路径、工具序列或策略不同，但仍然保持可验证和任务相关。工程上应区分 lexical diversity、semantic diversity、trajectory diversity 与 reward diversity。只看 Self-BLEU/字符串差异会把“措辞变化”误当成“行为探索”。

4. Best-of-N、Rejection Sampling 与 Search-Select-Train

4.1 Best-of-N 的核心：用计算换候选上限

Training Verifiers to Solve Math Word Problems 提出了生成大量候选并用 verifier 选最高分的范式；Self-Consistency 则从多条推理路径中按最终答案一致性聚合。这两类工作共同说明：单次 greedy 输出并不等于模型“知道的最好答案”，候选搜索能暴露模型潜在能力。[S6][S7]

```
BestOfN(q):
    candidates = [sample(policy, q) for _ in range(N)]
    scores = verifier_or_reward(q, candidates)
    return argmax(scores)
```

4.2 Llama 2 : Breadth 与 Iteration

Llama 2 把 rejection sampling 描述为：每个 prompt 从最近模型采样 K 个回答，用当前最佳 reward model 评分并选择最高分，再把选中样本作为新 gold 做梯度更新。其分析明确区分了 rejection sampling 的 breadth（同一 prompt 探索 K 个样本）与 PPO 的 depth（更新后立刻由新 policy 继续采样）。[S1]

另一个重要经验是“不要只保留最近一代候选”。Llama 2 早期只从上一轮样本袋选择时观察到部分能力回退，随后把更早迭代的 top-performing samples 重新纳入，改善了遗忘问题。[S1]

4.3 Llama 3 : 10-30 候选 + RM 选优 + 最新 Policy

Llama 3 在 human annotation prompts 上，每个 prompt 从最新 chat policy 通常采样 10-30 个输出，再由 reward model 选择最佳候选。后期还会使用 capability-specific system prompt 控制 tone/style/format。[S2]

这说明成熟 RS 已经不是单纯“增大 N”，而是由 policy checkpoint、system prompt、capability router、N、budget 和 RM 共同定义。

4.4 未被选中的候选也有价值

在 Llama 3 的多模态 post-training 描述中，rejection sampling 的未选候选会自然成为 on-policy rejected samples，用于 preference data；这是非常重要的“数据复用”设计：一次 generation job 可以同时服务 SFT gold 与 DPO negatives。[S2]

5. Self-Consistency、Verifier Search 与 Test-Time Compute 的启发

5.1 Self-Consistency：不一定要选“单条最高分轨迹”

Self-Consistency 对复杂推理采样多条不同 reasoning path，再对最终答案做一致性聚合；原论文在 GSM8K 等多个基准上显著优于 greedy CoT。[S7] 对后训练的启示是：Candidate Pool 可以被“聚合”而不仅仅是“排序”。

Selection 机制	需要什么信号	优势	局限	适合
RM Best-of-N	scalar reward	简单、通用	RM 偏差/Goodhart	helpfulness/style
Verifier Best-of-N	可验证 correctness	客观	只适合可验证任务	math/code
Majority / Self-Consistency	答案聚合	无需 RM	多数也可能错	唯一答案推理
Process Verifier	step-level score	更早识别坏路径	训练/调用成本高	长推理
Environment Success	task completion	与 Agent 目标一致	慢、稀疏	tool/agent

5.2 Compute-optimal：难题和值得搜索的题应该拿到更多预算

Snell 等对 test-time compute 的研究表明，不同 prompt 难度下最优计算分配策略不同，按 prompt 自适应分配搜索预算可以比固定 Best-of-N 更高效。[S8] 虽然这是 inference-time work，但对 post-training Response Generation 的直接启发是：N 与 max_tokens 应当是函数 $N(q, policy_state)$ ，而不是全局常量。

```
budget(q) = f(difficulty_estimate, reward_variance, pass_rate, uncertainty, product_value)
N(q), max_tokens(q), search_depth(q) <- budget(q)
```

可操作结论

把“每题 N=16”升级为“预算控制器”：容易题 N 小、前沿题 N 大、全错且不可学习的题暂时降权；动态采样本质上就是这种思想的第一步。

6. Reasoning RL : Group Rollout 成为基本训练单元

6.1 DeepSeek-R1 : 16 outputs / question 的组内学习

DeepSeek-R1-Zero 的 GRPO 对每个问题采样一组输出，并用组内 reward 的相对位置构造 advantage。公开训练细节包括：rollout temperature=1，每题 16 个输出；早期最大长度 32,768 tokens，8.2k step 后提高到 65,536；每个 rollout 大批次生成 8,192 个输出。[S4]

这意味着 Response Generation 直接定义了 advantage estimator 的统计性质：group size 太小，reward mean/std 噪声大；group 太大，token 和生成时间迅速上涨。

6.2 DAPO : Generation 与“有效梯度率”耦合

DAPO 指出，当一个 prompt 的 group 全对或全错时，GRPO 类方法可能得到零或低价值梯度。其 Dynamic Sampling 会过采样并过滤 accuracy=0 或 accuracy=1 的组，直到 batch 被具有有效组内差异的 prompt 填满。[S5]

公开设置中，DAPO 使用 prompt batch size=512、每 prompt 16 responses；最大 generation token 设为 20,480（其中 16,384 是期望上限，4,096 为 soft punishment cache）。[S5]

6.3 从“样本数”转向“有效 rollout 数”

指标	普通统计	更值得看的统计
数量	responses generated	non-degenerate groups / effective rollouts
准确率	mean reward	reward variance / mixed-success ratio
多样性	unique strings	unique reasoning/tool trajectories
长度	avg tokens	useful tokens before success / truncation rate
吞吐	tokens/s	effective learning signal / GPU-hour

核心观点

RL rollout engine 不应该只追求“生成更多 token”，而应该追求“产生更多能改变 policy 的高信息经验”。

7. Reasoning Budget、Response Length 与 Truncation

7.1 长度是“思考空间”，但不是能力本身

DeepSeek-R1-Zero 在 RL 训练中出现 response length 增长，并伴随反思、验证和替代方案探索。[S4] DAPO 也把 response length 与 validation accuracy 一起作为训练稳定性指标。[S5] 但长度增长不能被机械解释为 reasoning 变强：错误轨迹同样可以变长。

7.2 四种长度失败

失败	表现	原因	处理
Under-thinking	很快给最终答案但错	max tokens/stop 太紧或 policy 过早收敛	提高 budget；难度自适应
Truncation	正确过程在结尾被切断	hard max limit	监控 truncation；soft boundary
Over-thinking	正确后继续绕圈	缺少 stop incentive	stop reward / answer confidence
Degenerate long-tail	重复、乱码、模板循环	sampling/optimization 不稳	pattern filter + loss/reward shaping

7.3 DAPO 的 Overlong Reward Shaping 为什么重要？

如果把所有截断样本直接判成负 reward，一个本来合理但因长度上限被切断的推理也会被错误惩罚，产生 reward noise。DAPO 先实验 overlong filtering，再提出 soft overlong punishment，让长度接近上限时逐渐惩罚，而不是在边界处突然翻转。[S5]

Know-How

max_tokens 不是一个简单“显存参数”。它同时改变：搜索空间、截断率、reward 定义、batch 长尾、KV cache 占用、step wall-time。因此任何长度改动都应 与 reward/throughput 一起做联合 ablation。

8. Preference Response Generation : On-policy、Off-policy 与 Candidate Contrast

8.1 三种 candidate source

来源	定义	优点	缺点
On-policy	由当前/近期 policy 生成	贴近当前失败边界	质量可能较低；要频繁刷新
Near-policy	前一轮/相近 checkpoint	稳定且较新	仍存在一定 shift
Off-policy teacher pool	强模型/不同模型生成	质量和风格多样	离 student distribution 远

8.2 Tulu 3 : 多模型池 + 自身 SFT policy

Tulu 3 将一部分 prompt 转成 preference data：对每个 prompt 随机选择多个模型生成 responses，同时加入 Tulu 3 SFT 模型的 on-policy response，再用 LLM judge 从 helpfulness、instruction-following、honesty、truthfulness 等维度评分。其公开 ablation 报告加入 on-policy data 改善了聚合 DPO 下游表现。[S3]

8.3 为什么需要“难负样本”？

如果 rejected response 明显胡言乱语，DPO 只学会“不要明显差”；高价值 pair 应让 chosen/rejected 共享大量表面结构，只在事实、约束遵循、推理步骤或工具选择上存在关键差异。Response Generation 因此要支持 controlled corruption、near-miss generation、constraint perturbation 和 on-policy failure mining。

8.4 Preference Candidate Contract

```
candidate = {
  prompt_id, policy_id, policy_step, generator_family,
  response, token_count, logprob_summary, stop_reason,
  sampling_config, system_prompt_id, tool_trace?,
  judge_scores?, verifier_result?, safety_flags?
}
```

9. Agent Response Generation : 从 Completion 升级为 Environment Trajectory

Agent 后训练的生成单位不是单条 completion，而是状态-动作-观察序列。对 coding/search/tool agent，模型输出 Tool Call 会改变环境，环境结果再进入下一步推理。生成系统必须同时编排 LLM inference 与 tool/runtime。

```
s0 = task + repo/browser/tool state
a0 = policy(s0)           # text or tool call
o1 = env.step(a0)         # observation
s1 = append(s0, a0, o1)
...
τ = (s0, a0, o1, s1, a1, o2, ..., final)
```

9.1 Agent trajectory 的额外控制变量

变量	普通 Chat	Agent
Stop	EOS / max tokens	task success / tool budget / time budget / dead-loop
Length	output tokens	tokens + tool steps + environment latency
Diversity	wording/reasoning	action sequence / file/search path / recovery strategy
Reward	response score	task completion + tests + cost + safety
Failure	bad answer	invalid tool / env error / stale state / loop / partial completion

9.2 Llama 3 的工具经验提醒

Llama 3 报告指出，tool dialogs 往往包含多个 assistant message（调用工具、再对结果推理），因此其 human feedback 在 message level 采集；同时报告称对工具任务没有观察到 rejection sampling 的收益，所以没有沿用 RS，而是通过合成数据与逐步复杂化的人工协议来 bootstrap tool use。[S2]

Know-Why

“一种 Response Generation 技术在 Chat/Math 有效”并不意味着在 Agent/Tool 场景同样有效。环境反馈会改变搜索结构，必须做 stage × capability 的独立 ablation。

10. Rollout System : 为什么 Response Generation 会成为 RL 的吞吐瓶颈？

自回归生成存在强顺序依赖，长 CoT 又造成输出长度长尾。训练阶段可以大矩阵并行，而 rollout 需要逐 token 推进；因此现代 reasoning RL 常出现“生成比参数更新更慢”的系统结构。

10.1 PagedAttention / Prefix Sharing

vLLM 的 PagedAttention 通过分页式 KV cache 管理减少碎片并提高 batch capacity，原论文报告相较当时 serving baseline 有 $2-4\times$ throughput 改善。[S9] Llama 3 在 rejection sampling 中进一步利用 prompt KV pages 在同一 prompt 的多个 outputs 间共享，并配合最大长度 admission control，报告 RS 吞吐提升超过 $2\times$ 。[S2]

10.2 Training ↔ Generation 的模型状态切换

RL 每轮都需要“用当前 policy 生成，再更新 policy”。大模型下，训练分片与推理分片的并行策略不同，权重同步/reshard 成为核心成本。HybridFlow/verl 专门围绕 RLHF dataflow 和 actor 在 training/generation 之间的 reshard 设计系统抽象。[S10]

10.3 关键系统指标

指标	定义	为什么重要
Rollout tokens/s	总生成 token / wall time	基础吞吐
Prompt throughput	完成 prompt 数 / wall time	受 N 与长尾影响
P50/P95 trajectory length	轨迹长度分布	识别长尾阻塞
GPU utilization	生成/训练各阶段利用率	判断互等
Weight sync time	policy 更新→rollout 可见的延迟	影响 freshness
KV cache hit/share	前缀共享收益	Best-of-N 场景特别关键

11. Synchronous vs Asynchronous Rollout

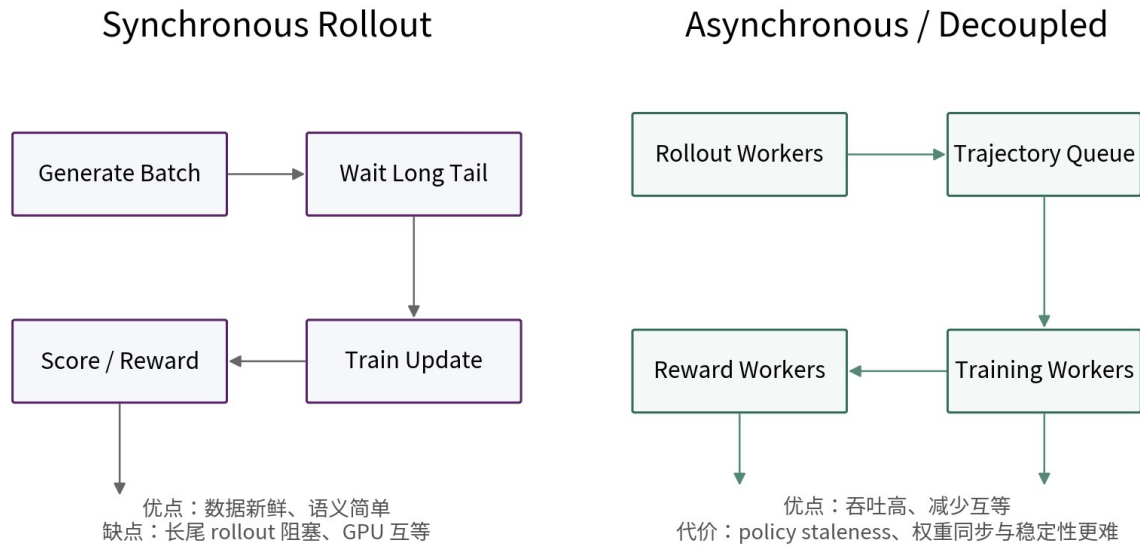


图 4 同步与异步 rollout 的核心权衡

11.1 同步模式

优点是所有 rollout 基本由同一个最新 policy 生成，训练语义清晰；缺点是 batch 必须等待最慢轨迹完成。长 CoT、工具调用和异常环境会放大 straggler 问题。

11.2 异步模式

ARaL 将 generation 与 training 完全解耦：rollout workers 持续生成，training workers 在收集够数据时更新，并通过控制 staleness 与专门的 PPO 变体保持稳定；其论文在数学与代码任务上报告最高 $2.57\times$ 的训练加速。[S11]

这种架构把“吞吐问题”转换成“数据新鲜度问题”：policy version k 生成的 trajectory 可能在 policy $k+d$ 时才被训练使用。

11.3 Staleness Contract

```
trajectory.policy_version = k
current_train_version = k + d
staleness = d

accept if staleness <= threshold
else: drop / importance-weight / replay-policy handling
```

Know-How：异步必须同时监控 staleness、importance ratio 与 reward drift；不要只看 tokens/s，否则会掩盖 off-policy 偏差。

12. 2025-2026 Frontier : Speculative Rollout 与 Generation Acceleration

随着 reasoning RL 的 rollout 长度上升，研究开始把 speculative decoding 从“线上推理加速”搬进 RL rollout。这里的关键要求是：不能为了提速破坏 target policy 的采样分布，否则训练数据语义会改变。

12.1 Lossless Speculation 的价值

2026 年关于 NeMo-RL 的系统研究将 speculative decoding 集成进同步/异步 RL rollout，并强调保持 target model 输出分布；在 8B reasoning post-training workload 中报告 $1.8\times$ rollout throughput，并模拟其与异步 RL 组合在更大规模上的潜在端到端收益。[S12]

12.2 为什么 Best-of-N 特别适合做系统优化？

同一 prompt 的 N 个候选共享长 prompt prefix，因此 KV cache/prefix sharing、batch scheduling、speculative draft reuse 都有很高复用空间。Response Generation 的“算法参数 N”与“系统优化空间”天然耦合。

加速手段	是否改变目标分布	主要收益	主要风险
Paged KV cache	否	扩大 batch / 减少碎片	调度复杂
Prefix KV sharing	否	同 prompt 多候选显著省显存	实现依赖
Speculative decoding	设计正确时否	减少 target forward 次数	acceptance rate 不稳
Async rollout	可能引入 off-policy	减少生成/训练互等	staleness
Lower precision inference	可能有轻微数值变化	吞吐/显存	reward distribution drift
Replay old rollouts	是	省生成成本	policy mismatch

13. Candidate Pool / Trajectory Store : 生产级数据契约

如果 Response Generation 只把“文本”写进 JSONL，后续几乎无法诊断为什么某一轮训练变差。成熟系统必须记录生成过程本身。

```
{
  "prompt_id": "...",
  "task_type": "math|code|agent|chat",
  "policy": {"model_id": "...", "step": 8200, "checkpoint_hash": "..."},
  "sampling": {"temperature": 1.0, "top_p": 0.95, "n": 16,
    "max_tokens": 32768, "seed": 1234},
  "response": {"text": "...", "tokens": 6842, "stop_reason": "eos"},
  "trajectory": {"tool_calls": [], "observations": [], "env_id": null},
  "statistics": {"logprob_mean": -0.42, "entropy_mean": 1.31},
  "evaluation": {"reward": 1.0, "verifier": "exact_match_v3"},
  "lineage": {"generator_job": "...", "created_at": "..."}
}
```

13.1 必须版本化的对象

对象	为什么
policy checkpoint	决定 on/off-policy 与可复现性
sampling config	T/top-p/N/max_tokens 改变候选分布
system prompt / chat template	会改变格式与行为先验
tool/environment version	Agent trajectory 结果依赖环境
verifier/reward version	同一 response 可能在新 verifier 下得分不同
stop/timeout policy	决定 truncated/failed 的语义

13.2 为什么要保存“没选中的候选”？

- 可以构造 preference rejected data。
- 可以离线重跑新 verifier / reward，避免重复昂贵 rollout。
- 可以分析 diversity、reward calibration、error taxonomy。
- 可以做 replay/offline RL，但必须保留 policy version 与 logprobs。

14. Response Generation 的 KPI：不要只看 tokens/s

维度	核心指标	解释
Coverage	prompt coverage / task coverage	有没有覆盖目标能力与长尾
Quality	mean/top reward, verifier pass rate	候选平均/上限质量
Diversity	semantic/trajectory diversity	是否真的探索不同策略
Signal	reward variance, mixed-success ratio	候选是否能产生学习梯度
Length	avg/P95 tokens, truncation rate	思考预算与长尾
Freshness	policy staleness	经验与当前 policy 的距离
Efficiency	effective rollouts/GPU-hour	真正的训练生产率
Stability	entropy trend, invalid/loop rate	是否出现 collapse/degeneration

14.1 推荐的“有效学习信号”仪表盘

```
mixed_success_rate = P(0 < #correct_in_group < N)
reward_variance = Var(r_1...r_N | q)
unique_strategy_rate = unique_semantic_or_action_paths / N
effective_rollout_rate = rollouts_used_for_gradient / rollouts_generated
compute_efficiency = reward_gain_or_eval_gain / GPU_hour
```

14.2 训练中应联合监控的四条曲线

DAPO 特别强调 response length、reward、generation entropy、mean probability 等中间量的联合监控。[S5] 单一 reward 上升并不能证明训练健康：可能同时伴随 entropy collapse、长度异常或 verifier shortcut。

实践规则

每次修改 N / T / max_tokens / stop / rollout architecture，至少同时观察：Eval、Reward、Length、Diversity/Entropy、Throughput、Truncation、Staleness。

15. 16 类常见失败模式与诊断

失败模式	症状	根因	优先修复
Candidate Collapse	N 条几乎一样	T 太低 / policy 熵塌缩	提高探索；监控 semantic diversity
Noise Explosion	大量不连贯	T/top-p 太激进	降低随机性；cap tail
All-Correct Groups	GRPO 梯度弱	任务太易	动态采样/难度上移
All-Wrong Groups	持续无正样本	任务超出当前能力	curriculum / teacher bootstrap
Reward Hacking Pool	高 reward 低人评	RM 被利用	多 verifier / audit / reward refresh
Length Inflation	错答案也越来越长	loss/reward length bias	Dr.GRPO/DAPO 类修正；长度诊断
Hard Truncation Noise	边界处 reward 异常	max token 直接判负	soft overlong shaping
Preference Too Easy	chosen/rejected 差太远	负样本太弱	near-miss/on-policy negatives
Off-policy Drift	DPO/RL 效果下降	候选来自旧/异构模型	提高 on-policy 比例；记录 lineage
Stale Async Rollout	训练不稳	生成落后多个 policy version	staleness gate / correction
Long-tail Straggler	GPU 等最慢 sample	长 CoT/环境延迟	async / repack / admission control
Tool Loop	反复调用同工具	stop/recovery 不完整	step budget + loop detector
Environment Flakiness	同轨迹 reward 不稳定	外部服务/测试不确定	环境版本化 + retry policy
Style Monoculture	所有答案一个模版	单 teacher/system prompt	多 generator / prompt steering
Data Waste	大量候选只丢弃	生成与下游脱节	复用为 negatives / replay / audit
Metric Gaming	tokens/s 上升但效果不变	只优化系统吞吐	改用 effective signal/GPU-hour

16. Response Generation Design Framework : 从 Objective 编译到 Config

16.1 六步法

步骤	问题
1. 定义下游	SFT target / preference contrast / RL experience / agent trajectory
2. 定义信号	RM、规则 verifier、process verifier、environment success、human
3. 定义搜索宽度	N、generator pool、temperature/top-p、system prompt variants
4. 定义搜索深度	max_tokens、max_steps、tool budget、timeout、stop policy
5. 定义 freshness	on-policy 目标、允许 staleness、checkpoint refresh cadence
6. 定义成本与验收	GPU-hour、effective rollout rate、eval gain、failure thresholds

16.2 配置模板

```

response_generation:
  stage: rlvr
  policy_source: latest_checkpoint
  prompts_per_rollout_batch: 512
  candidates_per_prompt: 16
  sampling:
    temperature: 1.0
    top_p: 0.95
    max_new_tokens: 16384
  dynamic_budget:
    enabled: true
    increase_if_reward_variance_gt: 0.15
    drop_if_group_all_correct: true
    defer_if_group_all_wrong: true
  stop:
    eos: true
    truncate_policy: soft_penalty
  lineage:
    store_logprobs: true
    store_policy_version: true
    store_all_candidates: true
  acceptance:
    require_parseable: true
    verifier_version: math_exact_v5

```

17. 五套可直接用的 Generation Recipe

Recipe A : 高质量 SFT Teacher Completion

- N=2-4: 不是为了大搜索, 而是为了降低单次 teacher 失误。
- 低-中温度; 输出后强 verifier / schema check。
- 保留失败原因, 不要把 prompt validity 和 response correctness 混在一起。

Recipe B : Preference Candidate Pool

- 混合 on-policy + strong off-policy teacher。
- 每 prompt 4-8 candidates; 控制长度差异, 避免 Judge 只学 length bias。
- 优先构造 near-miss rejected: 事实错一处、约束漏一项、工具选择次优。

Recipe C : Math/Code RLVR

- group size 8-16 起步; 用 pass-rate/reward variance 动态调预算。
- max_tokens 要和 truncation policy 联合设计。
- 过滤 verifier 不稳定样本; 保留所有 rollout 与 policy version。

Recipe D : Long-CoT Reasoning

- 监控 P50/P95 length、entropy、correct/incorrect length gap。
- 对全对组减少 rollout; 对中等成功率题增加 breadth。
- 把“更长”当探索资源, 不当 KPI。

Recipe E : Coding/Tool Agent

- 预算单位改成: tokens + tool calls + env seconds。
- trajectory 级 stop: task success、max steps、loop detector、timeout。
- 异步 rollout 前先定义 staleness contract; 工具/仓库环境必须版本化。

18. 成熟度模型：从“generate()”到 Adaptive Experience Controller

Level	系统特征	Response Generation 能力
L0 — Decode	固定 temperature/top-p	只生成单条 response
L1 — Candidate	固定 N + RM/Verifier	Best-of-N / rejection sampling
L2 — Stage-aware	SFT/DPO/RL 独立配置	on/off-policy、长度与候选策略分离
L3 — Online Rollout	policy version + RL engine	group rollout、tool/env、freshness
L4 — Adaptive Controller	基于难度/reward variance/成本动态分配	每 prompt 自适应 N、budget、generator、stop

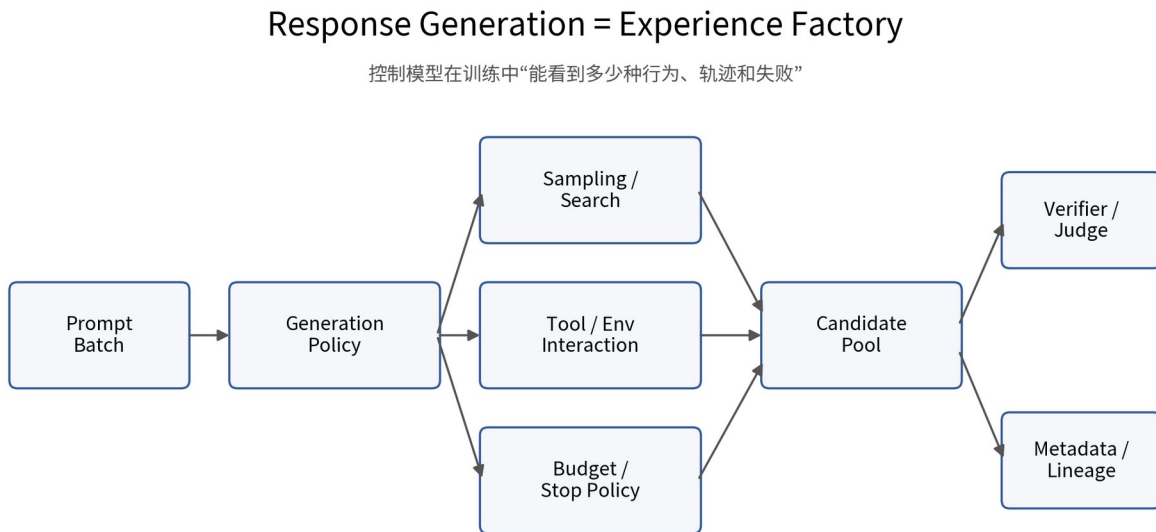
18.1 L4 的目标函数

```
maximize    Expected Learning Gain
subject to  GPU budget, token budget, environment budget, freshness constraint

control variables = {N(q), T(q), top_p(q), max_tokens(q), generator(q), stop(q)}
```

这时 Response Generation 已经不是一个“推理服务”，而是一个在线优化控制器：根据当前模型能力和训练信号，决定下一批经验应该怎样被制造。

19. 综合案例：从 Prompt 到 GRPO/DAPO Rollout 的完整数据流



Step 1 : Prompt Router

按 capability、difficulty、verifiability 分流；对于不可验证 general chat，不进入纯 RLVR path。

Step 2 : Policy Snapshot

冻结本 rollout iteration 使用的 policy checkpoint/version，明确 on-policy 语义。

Step 3 : Group Generation

对每个 prompt 生成 N 条候选；记录 token logprobs、stop reason、length、sampling config。

Step 4 : Verifier / Reward

对每条候选执行 exact match、unit tests、rule verifier 或 RM；保存原始细分信号。

Step 5 : Dynamic Gate

判断 group 是否 all-correct/all-wrong、是否 truncated、reward variance 是否足够；决定保留、补采、降权或 defer。

Step 6 : Train Buffer

把有效 trajectories 写入训练 buffer，附 policy version；异步系统还要记录 staleness。

Step 7 : Feedback

用 eval gain、reward variance、长度和成本更新下一轮的 N/max_tokens/task allocation。

系统闭环

Prompt Distribution 决定“训练什么”；Response Generation 决定“模型能探索到什么”；Verifier 决定“什么被视为好”；Optimizer 决定“如何把好行为写进参数”。四者必须联合设计。

20. 公的 Response Generation 对照表

系统/论文	Generation 设计	关键数字/经验	可迁移 Know-How
Llama 2 [S1]	Iterative rejection sampling + RM	K candidates; 温度随 RLHF 迭代重校准; 跨轮保留高质量样本	Search breadth 与 policy iteration 要一起设计
Llama 3 [S2]	Latest-policy RS + capability steering	通常 K=10-30; PagedAttention/prefix share 使 RS >2× throughput	RS 是模型、system prompt、RM、infra 的联合系统
Tulu 3 [S3]	Multi-model preference candidates	off-policy 多模型 + Tulu SFT on-policy; on-policy 有益	Candidate source portfolio 很重要
DeepSeek-R1 [S4]	GRPO long-CoT rollout	16 outputs/question; T=1; 长预算; 后阶段 T=0.7	rollout config 是训练状态变量
DAPO [S5]	Dynamic sampling + overlong shaping	512 prompts × 16; 过滤全对/全错; 20,480 max generation	优化 effective group, 而非 raw samples
Self-Consistency [S7]	多 reasoning paths + answer aggregation	多样路径投票	候选池可聚合而非只排序
vLLM [S9]	Paged KV cache	serving 2-4× throughput	Generation infra 决定 RL 可扩展性
AReAL [S11]	Fully async generation/training	最高 2.57× training speedup	吞吐与 staleness 是一对权衡
Speculative RL 2026 [S12]	Lossless speculative rollout	8B workload 1.8× rollout throughput	下一代优化重点是长 rollout 自回归瓶颈

21. 上线前检查清单 (Engineering Checklist)

目标与阶段

- ☐ 当前 generation job 是为 SFT、Preference 还是 RL 服务？
- ☐ 是否定义了 candidate/trajectory 的成功标准？
- ☐ 是否按 capability 使用不同 system prompt / generator？

采样与预算

- ☐ N、T、top-p 是否经过当前 checkpoint 的重新校准？
- ☐ 是否有 max_tokens/steps 与 soft truncation 方案？
- ☐ 是否按难度/reward variance 动态分配预算？

数据与新鲜度

- ☐ 是否记录 policy version、sampling config、seed、stop reason？
- ☐ on-policy/off-policy 比例是否明确？
- ☐ 异步 rollout 是否定义 staleness 阈值？

质量与信号

- ☐ 是否监控 mixed-success rate / reward variance？
- ☐ 是否保存未选候选供 preference/re-eval？
- ☐ Verifier/RM 是否版本化并做抗 reward hacking 审计？

系统

- ☐ 是否监控 P95 length / straggler？
- ☐ 是否利用 prefix KV sharing / continuous batching？
- ☐ 训练与生成的 GPU 分配是否基于实际 wall-time 调整？

Eval

- ☐ 改 generation 参数后是否同时跑能力、长度、风格、成本回归？
- ☐ 是否比较 effective learning signal/GPU-hour，而非只看 tokens/s？

22. 研究来源与阅读顺序

说明：正文中的 [S#] 对应下列公开论文/官方技术资料。优先使用论文与官方项目页；涉及 2025-2026 rollout systems 的结论按论文公开实验表述。

ID	来源	链接	本手册用途
S1	Touvron et al., Llama 2: Open Foundation and Fine-Tuned Chat Models (2023)	https://arxiv.org/abs/2307.09288	Rejection Sampling、温度与迭代采样的经典工程描述
S2	Grattafiori et al., The Llama 3 Herd of Models (2024)	https://arxiv.org/abs/2407.21783	10-30 候选 RS、PagedAttention/prefix sharing、tool generation
S3	Ai2, Tulu 3: The next era in open post-training (2024)	https://allenai.org/blog/tulu-3-technical	多模型 + on-policy preference response generation
S4	DeepSeek-AI, DeepSeek-R1 (v2, 2026)	https://arxiv.org/abs/2501.12948	16-rollout GRPO、长 CoT budget、stage-specific temperature
S5	Yu et al., DAPO (NeurIPS 2025)	https://arxiv.org/abs/2503.14476	Dynamic Sampling、Overlong Reward Shaping、rollout settings
S6	Cobbe et al., Training Verifiers to Solve Math Word Problems (2021)	https://arxiv.org/abs/2110.14168	generate-many + verifier selection
S7	Wang et al., Self-Consistency Improves Chain of Thought Reasoning (2022)	https://arxiv.org/abs/2203.11171	多 reasoning paths + consistency aggregation
S8	Snell et al., Scaling LLM Test-Time Compute Optimally (2024)	https://arxiv.org/abs/2408.03314	按 prompt 难度自适应分配 search compute
S9	Kwon et al., Efficient Memory Management for LLM Serving with PagedAttention (2023)	https://arxiv.org/abs/2309.06180	vLLM/PagedAttention 与高吞吐生成
S10	Sheng et al., HybridFlow: A Flexible and Efficient RLHF Framework (2024)	https://arxiv.org/abs/2409.19256	训练/生成 dataflow、resharding、verl
S11	Fu et al., AReaL: A Large-Scale Asynchronous RL System for Language Reasoning (2025)	https://arxiv.org/abs/2505.24298	generation-training 解耦与 staleness
S12	Iso et al., Accelerating RL Post-Training Rollouts via System-Integrated Speculative Decoding (2026)	https://arxiv.org/abs/2604.26779	lossless speculative rollout 加速
S13	Lightman et al., Let’s Verify Step by Step (ICLR 2024)	https://arxiv.org/abs/2305.20050	process reward / verifier 对候选筛选的影响
S14	Liu et al., Understanding R1-Zero-Like Training: A Critical Perspective (2025)	https://arxiv.org/abs/2503.20783	GRPO length bias 与 Dr. GRPO

推荐阅读顺序

先读 S1/S2 建立 rejection sampling 与迭代生成直觉 → S4/S5 理解 reasoning RL 的 group rollout → S3 理解 preference candidate source → S9/S10/S11/S12 理解 rollout 系统工程 → S6/S7/S8/S13/S14 补充 verifier/search/selection 与偏差。

最终 Know-Why / Know-How

Know-Why	Know-How
Response Generation 决定训练可见的行为搜索空间	按 stage 设计不同 candidate distribution，而不是共用 generation config
多样性是为了产生可选择/可学习差异	监控 semantic/trajectory/reward diversity，不只看文本差异
on-policy 能暴露当前 policy 的真实失败边界	记录 policy version，控制 freshness/staleness
长推理需要预算，但长不是目标	把 max_tokens 与 truncation/reward/throughput 联合优化
RL 的有效样本是“有梯度的 group”	用 mixed-success/reward variance 驱动动态采样
系统吞吐决定能否扩大搜索	KV sharing、continuous batching、async、speculation 与训练算法协同
最终目标是训练收益而不是生成数量	用 effective learning signal/GPU-hour 做核心效率指标